

Chapter 40

Embedded MRAM (F16ASM)

40.1 Chip-specific F16ASM information

40.1.1 MRAM partition

MRAM partitions are as described below:

Description	MRAMC Block	MRAM Address	MRAM Partition	Partition size KB
Code MRAM	Code 0	Lower	Partition 0	2048
	Code 0	Upper	Partition 1	
	Code 1	Lower	Partition 2	
	Code 1	Upper	Partition 3	
	Code 2	Lower	Partition 4	
	Code 2	Upper	Partition 5	
	Code 3	Lower	Partition 6	
	Code 3	Upper	Partition 7	
	Code 4	Lower	Partition 8	
	Code 4	Upper	Partition 9	
	Code 5	Lower	Partition 10	
	Code 5	Upper	Partition 11	
	Code 6	Lower	Partition 12	
	Code 6	Upper	Partition 13	
	Code 7	Lower	Partition 14	
	Code 7	Upper	Partition 15	
Config/Boot/Data MRAM	Data 0	Lower	Partition 16	256
Secure Code A MRAM	Secure 0	Upper	Partition 17	256
Secure Code B MRAM	Secure 1	Lower	Partition 18	256
Secure Data/SBAF MRAM	Secure 2	Upper	Partition 19	256

40.1.2 Address space of Code MRAM and Data MRAM

For details related to address space of Code MRAM and Data MRAM, see S32K56x memory map sheet (S32K56x_memory_map.xlsx) attached to the document for details.

40.1.3 Reset values

Table 194. Reset values

Register and field	Reset value on this chip (hex)
LOCK0[LPLOCKL]	FF
LOCK0[LPLOCKU]	FF
LOCK1[OLOCKL]	1
LOCK1[OLOCKU]	1
LOCK1[SPLOCKL]	3
LOCK1[SPLOCKU]	3
LOCK1[UTLOCKL]	1
LOCK1[UTLOCKU]	1
MCRS[SIZE]	6
MCRS[SSIZE]	2

40.2 Overview

F16ASM is [MRAM](#) that serves as electrically writable and readable nonvolatile memory. You can use it for instruction or data storage, particularly in embedded automotive and safety applications.

40.2.1 Block diagram

The following figure shows the F16ASM functional units:

- [CMMU](#)
- Analog components, comprising:
 - An internal oscillator
 - Generated voltages
 - Optional analog-test interface to external test modules
- The MRAM core

as well as other chip components (such as [MRAMC](#)) that F16ASM interacts with.

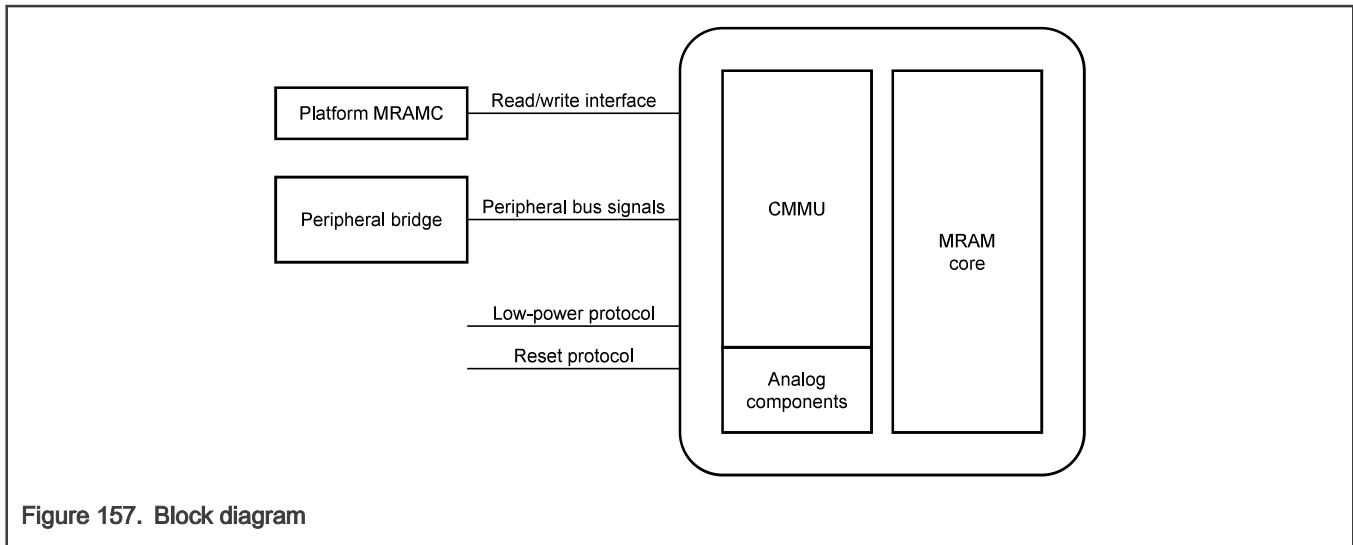


Figure 157. Block diagram

40.2.2 Features

- 8–40 MB in large partition space; the large partition space is always 2 MB per partition
- 512 KB or 1 MB in small partition space (optional, no small partitions); the small partition space is always 256 KB per partition (if present)
- UTest row, comprising 2 KB of MRAM per partition, in an independent space
- **OTP** space for security storage and boot memory
 - 32 KB for each small partition
 - 4 KB for each large partition
- 256-bit read page
- Write size of 8–256 bits, combined with 1024 bits for memory load and the following write options:
 - Direct write (of smaller than 256-bits) using MRAMC write buffers and [RMW](#)
 - 256-bit (page) write using MRAMC write buffers and RMW (if applicable)
 - 1024-bit Quad-page write using MRAMC write buffers with special mode and restrictions on address
- Multiple reads in parallel (including different partitions and the UTest row) using [Tri-read](#)
- Read-while-write operation with:
 - Partition granularity (2 MB or 256 KB MRAM space)
- Low-power mode
- Safety-specific features:
 - Embedded write-retry mechanism with post-write verification
 - Write-lock capability for each large and small partition
 - Reset error handling
 - ECC:
 - Single-bit correction
 - Double-bit correction
 - Triple-bit correction
 - Quad-bit detection

- ECC algorithm with "all 0s" and "all 1s" invalid for MRAM array
- EDC after ECC
- Address encode
- Write address integrity check at F16ASM boundary and at MRAM array boundary
- Reference recovery and refresh
- Reference and sense amp trim check mechanism using a dedicated UTest row
- Triple-voted flops for safety-critical memory functions (for example, internal trimming, redundancy, and mode control)
- Parity on internal repair storage and communication
- ECC on internal repair storage and communication
- UTest mode (user-accessible test modes included for chip-safety concept) including:
 - Array integrity
 - ECC logic check
 - EDC after ECC logic check
 - Address encode logic check
- LVD and HVD detection (external to MRAM)

40.3 Functional description

40.3.1 Component instances

The following table describes the number of components within F16ASM.

Table 195. Component instances

Component	Instances
CMMU	One
Analog	Two or more
Memory array	Variable number of small and large partitions: • 0, 2, or 4 small partitions, each 256 KB in size and containing 32 KB of OTP • 4–20 large partitions, each 2 MB in size and containing 4 KB of OTP

40.3.2 F16ASM array memory map

Table 196. F16ASM array memory map

Byte address	Contents	Size	Additional information
0–1F_FFFFh	First MRAM large partition	2 MB	Applies to upper and lower address
0–7F_FFFFh	MRAM large partition range	8 MB	Applies to upper and lower address for a total of 16 MB
0–1FF_FFFFh	MRAM large partition range	32 MB	Applies to upper and lower address for a total of 64 MB

Table continues on the next page...

Table 196. F16ASM array memory map (continued)

Byte address	Contents	Size	Additional information
0–3_FFFFh (with small partition selected)	First MRAM small partition	256 KB	Applies to upper and lower address; 2 small partitions
0–7_FFFFh (with small partition selected)	First MRAM small partition	512 KB	Applies to upper and lower address; 4 small partitions
Partition start plus offset equal to 0–7FFh with UTest row selected	UTest row	2 KB	Offset is overlaid with each partition
Partition start plus offset equal to 0–FFFh with OTP selected	Large partition OTP	4 KB	Offset is overlaid with each large partition
Partition start plus offset equal to 0–7FFFh with OTP and small partition selected	Small partition OTP	32 KB	Offset is overlaid with each small partition

40.3.3 Modes of operation

Table 197. Modes of operation

Mode	Description
User	Normal operation in which you can read and write registers and the memory array. This is the default F16ASM mode.
Low-Power	Turns off all DC current sources within F16ASM. This mode produces the lowest current consumption that F16ASM can achieve.
UTest	Exposes a portion of the NXP factory-test modes to you. This mode is protected but accessible.

In the unlikely event that you request a combination of User mode OTP commit, Low-Power mode or UTest mode simultaneously, priority will be given in the order listed.

40.3.3.1 User mode

In this mode, you can read from and write to the embedded MRAM and OTP.

40.3.3.1.1 MRAM read

This is the only operation that allows you to read the MRAM array.

One read of the MRAM array returns 256 bits (1 page).

You can read MRAM (and OTP) while clock switching occurs (glitch-less clock switch). Before clock switching occurs, you must configure [MCR\[RWSC\]](#) for the faster frequency. You can also change [MCR\[RWSC\]](#) during a read operation, because F16ASM latches the [MCR\[RWSC\]](#) value at the beginning of the read request.

F16ASM allows you to perform MRAM array reads while MRAM writes are ongoing (read-while-write to different partitions).

The F16ASM registers are always available for reading in User mode.

Register reads return 32 bits of data. Attempting a register read of unmapped or unused register address spaces returns all zeroes and F16ASM returns a transfer error.

Attempting an array read from an invalid location produces indeterminate data. Invalid locations occur when addressing is done to partitions that do not exist based on the number of partitions in that configuration (out-of-range addresses).

40.3.3.1.2 OTP read

This is the only operation that allows you to read the OTP. MRAMC performs OTP reads in a similar way to how it performs MRAM reads.

One read of the OTP returns 256 bits (1 page).

F16ASM:

- Allows you to perform OTP reads while MRAM writes are ongoing (read-while-write to different partitions).
- Does not support tri-reads with OTP read requests.
- Will automatically enforce a minimal settling time equivalent to approximately the number of read wait state clocks between OTP reads and MRAM reads.

40.3.3.1.3 MRAM write

In the embedded MRAM, the typical memory-array write operations are 256 bits (1 page) in size. You can write data in smaller sizes—byte, word, and double-word—by using the internal F16ASM read-modify-write feature. The MRAM also supports quad-page writes for memory-loading purposes, in which data of four logically adjacent addresses (same row) can be provided for increased write throughput. Quad-page writes are not allowed on the UTest row and may result in `MCRS[WRE] = 1`. MRAMC may block MRAM writes, which may affect error reporting.

All MRAM writes include a verify read to ensure correct data is written, with appropriate error reporting.

If an MRAM location report errors (`MCRS[EER] = 1` or `MCRS[WRE] = 1`) you can perform a page write or quad-page write to that location in attempt to recover from the error. This is required due to the read-modify-write feature of the MRAM.

You can write to MRAM (and OTP) while clock switching occurs (glitch-less clock switch).

F16ASM allows you to perform MRAM array writes while MRAM reads are ongoing (read-while-write to different partitions, including the UTest row).

The F16ASM registers are always available for writing in User mode.

Register writes are 32 bits in size. Attempting a register write to unmapped or unused register address spaces has no effect. If you attempt an array write to an invalid location, F16ASM ignores the attempt and returns a transfer error.

40.3.3.1.4 OTP write

F16ASM does not allow you to perform OTP reads, MRAM reads or MRAM writes when you request or execute an OTP-write operation.

In the embedded MRAM, the OTP-write operation is 256 bits (1 page) in size.

40.3.3.1.5 OTP commit

An OTP commit is a special type of OTP-write operation that commits the written data permanently to memory.

F16ASM does not allow you to perform OTP reads, MRAM reads or MRAM writes when you request or execute an OTP-commit operation.

F16ASM does not accept a OTP commit request and will not make `MCRS[OMR] = 1`, until all of the following occur:

- `UT0[UTE]` is 0.
- Ongoing OTP reads, MRAM reads and MRAM writes that you requested are completed.
- MRAM is not in Low-Power Sleep mode (`MCRS[LPR] = 0` and `MCRS[LPA] = 0`).

40.3.3.1.6 Reference and sense amplifier trim safety check

The UTest row is a required part of the safety mechanism for checking the trims of the reference and sense amplifiers.

One time, prior to doing the following steps, you must write unique and known data (data n) or (~data n) to all UTest row locations as listed below.

The following table describes the procedure for you to perform this safety check. You must perform this procedure during each safety interval for each partition that you need to check. In summary, you perform the following tasks for each small and large partition:

Table 198. Reference and sense amplifier trim safety check

Step	Action
1	Read UTest offset byte address 0h and confirm value read is data0.
2	Read UTest offset byte address 20h and confirm value read is data0.
3	Read UTest offset byte address 40h and confirm value read is ~data0.
4	Read UTest offset byte address 60h and confirm value read is ~data0.
5	Read UTest offset byte address 380h and confirm value read is data1.
6	Read UTest offset byte address 3A0h and confirm value read is data1.
7	Read UTest offset byte address 3C0h and confirm value read is ~data1.
8	Read UTest offset byte address 3E0h and confirm value read is ~data1.
9	Read UTest offset byte address 400h and confirm value read is data2.
10	Read UTest offset byte address 420h and confirm value read is data2.
11	Read UTest offset byte address 440h and confirm value read is ~data2.
12	Read UTest offset byte address 460h and confirm value read is ~data2.
13	Read UTest offset byte address 780h and confirm value read is data3.
14	Read UTest offset byte address 7A0h and confirm value read is data3.
15	Read UTest offset byte address 7C0h and confirm value read is ~data3.
16	Read UTest offset byte address 7E0h and confirm value read is ~data3.
17	Repeat the preceding steps for each large and small partition. If all reads return expected data, then the safety check passes.

40.3.3.2 Low-Power Sleep mode

In this mode:

- The analog components of F16ASM shut down. This reduces F16ASM current.
- You cannot read from the memory array.
- You cannot write to the memory array or registers.
- You cannot change to another mode, except for returning to the immediately preceding mode.
- You can reset.

To enter this mode:

Table 199. Entering Low-Power Sleep mode

Step	Purpose	Programming
1	Request entry into Low-Power Sleep mode.	Wait until MCRS[LPR] = 1.
2	Allow time for F16ASM to accept your request and enter Low-Power Sleep mode. Low Power request will not be acknowledged until UT0[UTE] = 0, and ongoing OTP reads, MRAM reads, MRAM writes and OTP commits you requested are completed (MCRS[OMR] = 0 and MCRS[OMA] = 0).	Wait until MCRS[LPA] = 1.

40.3.3.3 UTest mode

This is a diagnostic mode that includes procedures that you can use for a safety concept. UTest mode allows you to perform specific tests that check the integrity of the embedded memory.

During array integrity or reference recovery and refresh, MRAM reads and writes through the MRAMC are not allowed while the operation is ongoing ([UT0\[AID\]](#) = 0). For other UTest mode operations, write, tri-read and read-while-write are supported.

After enabling UTest mode, and the completion of the desired procedure(s), you must return to User mode, and exit UTest mode by writing [UT0\[UTE\]](#) = 0.

40.3.3.3.1 Enabling UTest mode

Table 200. Enabling UTest mode

Step	Purpose	Programming
1	Ensure that a low-power request is not in process.	Wait until MCRS[LPR] = 0.
2	Ensure that an OTP commit is not in progress.	Wait until the OMR and OMA fields in Module Configuration Status (MCRS) are both 0.
3	Eliminate any errors to ensure robust software and procedure execution.	Examine all flags in Module Configuration Status (MCRS) and Read Address Logging (RADR) and take appropriate action to clear any flag that is set.
4	Provide the password (F9F9_9999h) that F16ASM needs to enter UTest mode.	Use a 32-bit write to write F9F9_9999h to UTest 0 (UT0) .
5	Allow time for F16ASM to accept your password and enter UTest mode.	Wait until UT0[UTE] = 1.

40.3.3.3.2 Array integrity self-check

F16ASM checks array integrity using a predefined address sequence (based on [UT0\[AIS\]](#)), and this operation executes on the selected partition.

The data to be read is your specific code that you write into the memory, and the correct MISR signature is calculated based on that code.

After the operation completes, you can check the results of the reads by reading the MISR value. This allows you to determine whether F16ASM observed an incorrect read or ECC detection. F16ASM calculates the array-integrity MISR value after ECC detection and correction. Array integrity requires that you write appropriate values to the RWSC and RWSL fields in [Module Configuration \(MCR\)](#) to match the system frequency that you are using.

To perform an array-integrity self-check:

40.3.3.3.3 Breakpoint processing

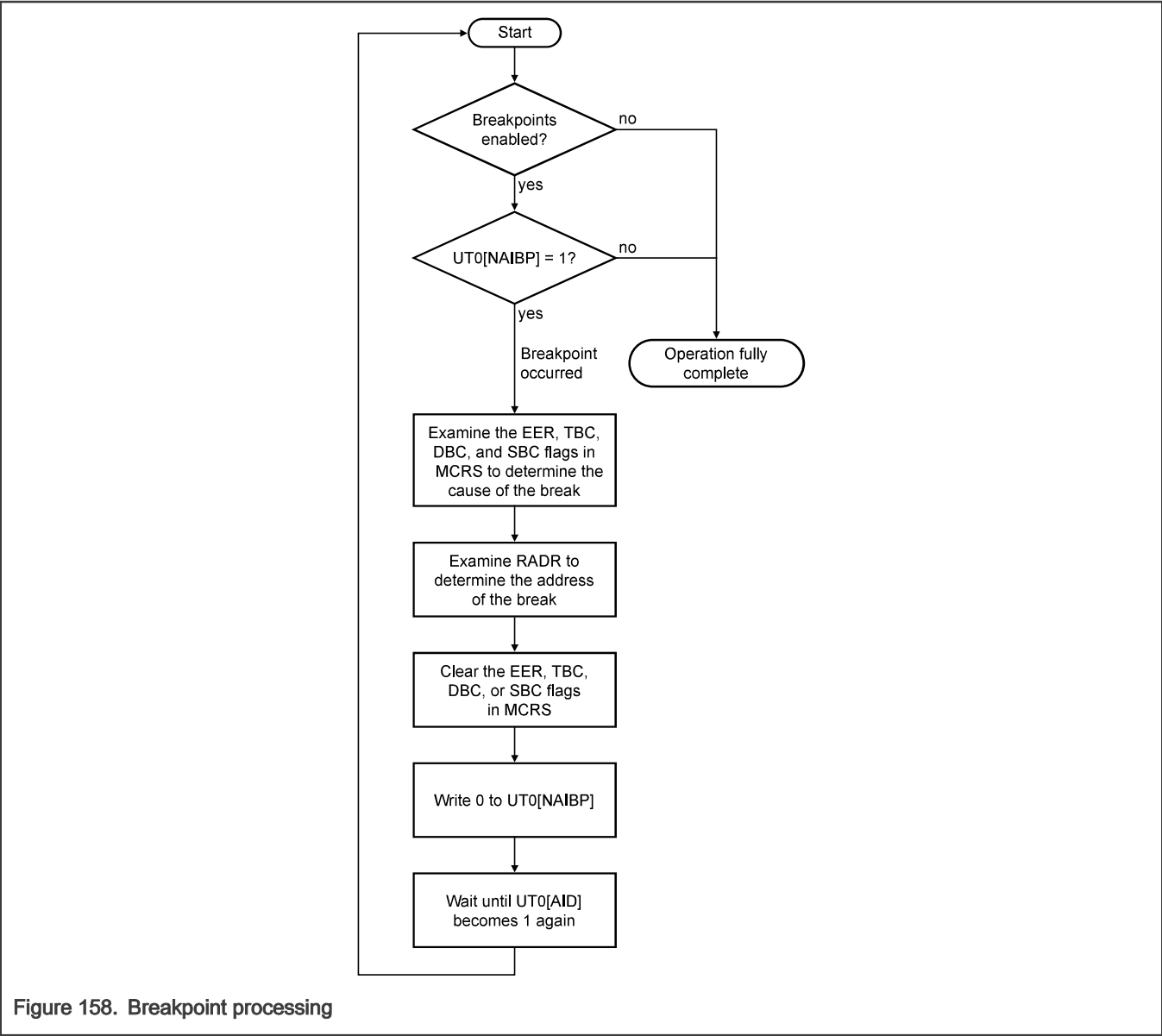


Figure 158. Breakpoint processing

40.3.3.3.4 ECC logic check

You can check the ECC logic by providing to-be-read data (in [UTest Data n \(UD0 - UD7\)](#), [UTest Data 8 \(UD8\)](#), or both), then reading the array to ensure expected results.

In the event of an ECC error or ECC correction, during the ECC logic check ($UT0[EIE] = 1$), [Read Address Logging \(RADR\)](#) is not loaded, and the address tagged to receive the ECC logic check values is preserved.

To perform an ECC logic check:

Table 202. ECC logic check

Step	Purpose	Programming
1	Enter UTest mode.	See Enabling UTest mode .

Table continues on the next page...

Table 202. ECC logic check (continued)

Step	Purpose	Programming
2	Enable the inversion of ECC data.	Write 1 to UT0[EIE] .
3	Provide data and check-bit values to read. You can simulate single-, double-, or triple-bit corrections, as well as quad-bit detections by properly selecting data and check-bit combinations.	Write the data to UTest Data n (UD0 - UD7) , UTest Data 8 (UD8) , or both.
4	Specify the page address to receive data that you provided in step 3.	Write the address to Read Address Logging (RADR) .
5	Read the array.	Submit an array-read request to the MRAMC.
6	Determine the status of the read operation.	Examine the EER, TBC, DBC, and SBC flags in Module Configuration Status (MCRS) .
7	Disable the inversion of ECC data.	Write 0 to UT0[EIE] .

40.3.3.3.5 EDC-after-ECC logic check

You can check the EDC-after-ECC logic by providing to-be-read data (in [UTest Data n \(UD9 - UD16\)](#), [UTest Data 17 \(UD17\)](#), or both), then reading the array to ensure expected results.

To perform an EDC-after-ECC logic check:

Table 203. EDC-after-ECC logic check

Step	Purpose	Programming
1	Enter UTest mode.	See Enabling UTest mode .
2	Enable the EDC-after-ECC data input.	Write 1 to UT0[EDIE] .
3	Provide data and check-bit values to read.	Write the data to UTest Data n (UD9 - UD16) , UTest Data 17 (UD17) , or both.
4	Specify the page address to receive data that you provided in step 3.	Write the address to Read Address Logging (RADR) .
5	Read the array.	Submit an array-read request to the MRAMC.
6	Determine the status of the read operation.	Examine the MCRS[EEEE] flag.
7	Disable the EDC-after-ECC data input.	Write 0 to UT0[EDIE] .

40.3.3.3.6 Address-encode logic check

You can check the address-encode logic by inverting the address-encode information from the memory array (in [UA0\[AEI\]](#), [UA1\[AEIC\]](#), or both), then reading the array to ensure expected results.

To perform an address-encode logic check:

Table 204. Address-encode logic check

Step	Purpose	Programming
1	Enter UTest mode.	See Enabling UTest mode .
2	Enable the inversion of the address-encode value.	Write 1 to UT0[AEIE] .

Table continues on the next page...

Table 204. Address-encode logic check (continued)

Step	Purpose	Programming
3	Provide the address bit or bits to invert.	Write the bit or bits to UA0[AEIS] , UA0[AEI] , UA1[AEIC] , or all.
4	Provide the address bit or bits to invert to the MRAMC.	Write the bit or bits to UA1[AEIO] .
5	Specify the page address to receive the inverted address that you provided in step 3.	Write the address to Read Address Logging (RADR) .
6	Read the array.	Submit an array-read request to the MRAMC.
7	Determine the status of the read operation.	Examine the MCRS[AEE] flag.
8	Disable the inversion of the address-encode value.	Write 0 to UT0[AEIE] .

40.3.3.3.7 Reference recovery and refresh

You can recover and refresh MRAM references if a known magnetic overexposure or an array detection corruption event occurs. Reads and writes of the MRAM are not allowed during a recover-and-refresh operation. Reference recovery and refresh can not be aborted or suspended. Reference recovery execution time is dependent on total density, and is up to 20 ms/MB.

To perform a recover-and-refresh operation:

Table 205. Reference recovery and refresh

Step	Purpose	Programming
1	Enter UTest mode.	See Enabling UTest mode .
2	Enable the operation.	Write 1 to UT0[RRE] .
3	Start the operation.	Write 1 to UT0[AIE] .
4	Allow time for F16ASM to complete the operation.	Wait until UT0[AID] becomes 1.
5	Determine the status of the read reference recovery and refresh operation.	Examine the UT0[RRER] flag.
6	Disable the operation.	Write 0 to the RRE and AIE fields in UTest 0 (UT0) .

40.3.4 Multiple input signature register (MISR)

40.3.4.1 Calculation

The MISR polynomial is:

$$x^{289} + x^7 + x^6 + x^5 + x^4 + x^2 + 1$$

Figure 159. MISR polynomial

F16ASM calculates the MISR according to the following process:

1. Apply the XOR logical operator to the previous MISR value and the new data.
2. Apply the XOR logical operator to the most-significant bit (288) and the input of MISR bits 7, 6, 5, 4, 2, and 0.
3. Shift the result of the XOR logical operator left.

For example, after a read operation, F16ASM calculates MISR[7] by taking the previous MISR[6] and applying the XOR logical operator to the new data RDATA[7] as well as MISR[288], then shifting left to create the new MISR[7]. The inclusion of MISR[288] is based on the polynomial chosen for F16ASM, and only applies to a subset of MISR calculations.

40.3.4.2 Data alignment

Table 206. Data alignment

MISR bit numbers	Value
255–0	Read data
283–256	ECC read data
287–284	All 1s
288	ECC error

40.3.4.3 Access

You can seed the MISR with any value by writing to [UTest Multiple Input Signature n \(UMISR0 - UMISR8\)](#) and [UTest Multiple Input Signature 9 \(UMISR9\)](#). You can always write to these registers prior to an array integrity operation. NXP recommends that you do not write to these registers during an array-integrity operation (including suspend or breakpoint), even though F16ASM allows it. Writing to these registers during an array-integrity operation may alter the final signature in an unpredictable way.

During address sequencing, if a read operation extends into an invalid address location—in other words, greater than the maximum address for a given array size—then F16ASM still reads from the array, but the results from the array read are not deterministic. In this scenario, the MISR is not recalculated, and the previous value is retained. If you read the MISR during an array-integrity operation ([UTO\[AIE\]](#) = 1 and [UTO\[AID\]](#) = 0), F16ASM returns all bits of the MISR as 0. After you write 0 to [UTO\[AIE\]](#) (with [UTO\[RRE\]](#) = 0), F16ASM changes all bits of the MISR to 0.

40.3.5 Clocking

This module has no clocking considerations.

40.3.6 Reset

A reset is the highest-priority operation for F16ASM. It terminates all other operations. F16ASM uses reset to initialize registers to their default reset values. If F16ASM is executing a write operation when a reset occurs, the operation aborts. Reset aborts all operations and forces F16ASM into User mode, ready to receive accesses.

After reset occurs, [MCRS\[DONE\]](#) becomes 0 and remains 0 during reset and reset recovery. At the end of reset recovery, [MCRS\[DONE\]](#) becomes 1.

During reset recovery:

- You can read registers, and when [MCRS\[DONE\]](#) is 0, all other registers will also read 0.
- You cannot write to registers until [MCRS\[DONE\]](#) is 1, indicating that reset recovery is complete.

Resetting during a write operation leaves the MRAM bits being modified in an indeterminate data state. You can recover from this by repeating the write to the affected location.

40.3.7 Interrupts

This module has no interrupts.

40.4 External signals

F16ASM has no external signals.

40.5 Initialization

F16ASM initializes itself during reset with no action required from you. See [Reset](#) for details.

40.6 F16ASM register descriptions

This section presents the F16ASM memory-mapped registers. For information about the memory array, see [F16ASM array memory map](#).

In the register memory-map space, the following addresses are reserved. If you try to access those addresses, F16ASM returns a transfer error.

- 8h
- 90h
- 9Ch
- 100–7FCh

40.6.1 F16ASM memory map

MRAM base address: 404A_0000h

Offset	Register	Width (In bits)	Access	Reset value
0h	Module Configuration (MCR)	32	RW	See section
4h	Module Configuration Status (MCRS)	32	RW	See section
Ch	Read Address Logging (RADR)	32	RW	0000_0000h
10h	Write Address Logging (WADR)	32	R	0000_0000h
14h	Partition Lock 0 (LOCK0)	32	R	See section
18h	Partition Lock 1 (LOCK1)	32	R	See section
1Ch	UTest 0 (UT0)	32	RW	0000_0001h
20h - 40h	UTest Multiple Input Signature n (UMISR0 - UMISR8)	32	RW	0000_0000h
44h	UTest Multiple Input Signature 9 (UMISR9)	32	RW	0000_0000h
48h - 64h	UTest Data n (UD0 - UD7)	32	RW	0000_0000h
68h	UTest Data 8 (UD8)	32	RW	0000_0000h
6Ch - 88h	UTest Data n (UD9 - UD16)	32	RW	0000_0000h
8Ch	UTest Data 17 (UD17)	32	RW	0000_0000h
94h	UTest Address 0 (UA0)	32	RW	0000_0000h
98h	UTest Address 1 (UA1)	32	RW	0000_0000h

40.6.2 Module Configuration (MCR)

Offset

Register	Offset
MCR	0h

Function

Controls aspects of F16ASM operation.

Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	OWC	Reserved		Reserved	WPL	Reserved										
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	Reserved								RWSL	Reserved			RWSC			
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	u ¹	u	u	u

1. This reset value is set at the factory. Reset value is set to max RWSC specified in RWSC tables. RWSC = 6 for configurations up to 13 MB. RWSC = 9 for configurations 16 MB up to 41 MB. RWSC = 10 for configurations 48 MB and larger.

Fields

Field	Function
31 OWC	OTP Write Commit Indicates the status of the request to commit the OTP space. F16ASM allows MRAMC to initiate a request when both of the following conditions are true: • F16ASM has received and acknowledged a request for an OTP commit (MCRS[OMR] = MCRS[OMA] = 1). • MCRS[DONE] = 1. F16ASM automatically changes OWC back to 0 after the OTP write-commit operation is completed and coincides with MCRS[DONE] becoming 1. 0b - No request 1b - Initiate request
30 —	Reserved

Table continues on the next page...

40.6.3 Module Configuration Status (MCRS)

Offset

Register	Offset
MCRS	4h

Function

Indicates F16ASM status.

Diagram

Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	EER	TBC	DBC	SBC	Reserved	AEE	EEE	RE	WRE	WCE	WRP	WAE	WTE	RSE	Reserved	DONE
W	W1C	W1C	W1C	W1C		W1C	W1C		W1C	W1C	W1C	W1C	W1C			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	REWSC				LPA	LPR	OMA	OMR	Reserved		SSIZE		SIZE			
W																
Reset	0	0	0	0	0	0	0	0	0	0	u ¹	u	u ¹	u	u	u

1. This reset value is set at the factory. See the chip-specific F16ASM information for the reset value on this chip.

Fields

Field	Function
31 EER	ECC Event Error Status Flag Indicates whether a noncorrectable ECC error occurred during a previous read. When an MRAM-space read operation contains a noncorrectable ECC error (quad-bit detection), F16ASM: - Sets the EER flag. - Captures the corresponding address in Read Address Logging (RADR). NOTE This field behaves differently for register reads and writes. When reading 0b - Reads occur normally 1b - ECC error occurred When writing 0b - No effect 1b - Clear the flag

Table continues on the next page...

Table continued from the previous page...

Field	Function
	0101b - Reserved. 0110b - 32 MB 0111b - Reserved. 1000b - 40 MB 1001b - Reserved. 1010b - 48 MB (expansion) 1011b - Reserved. 1100b - 56 MB (expansion) 1101b - Reserved. 1110b - 64 MB (expansion) All other values are reserved.

40.6.4 Read Address Logging (RADR)

Offset

Register	Offset
RADR	Ch

Function

Provides the following for read operations:

- Error status
- Correction status
- Controls for UTest address-tagging features

Diagram

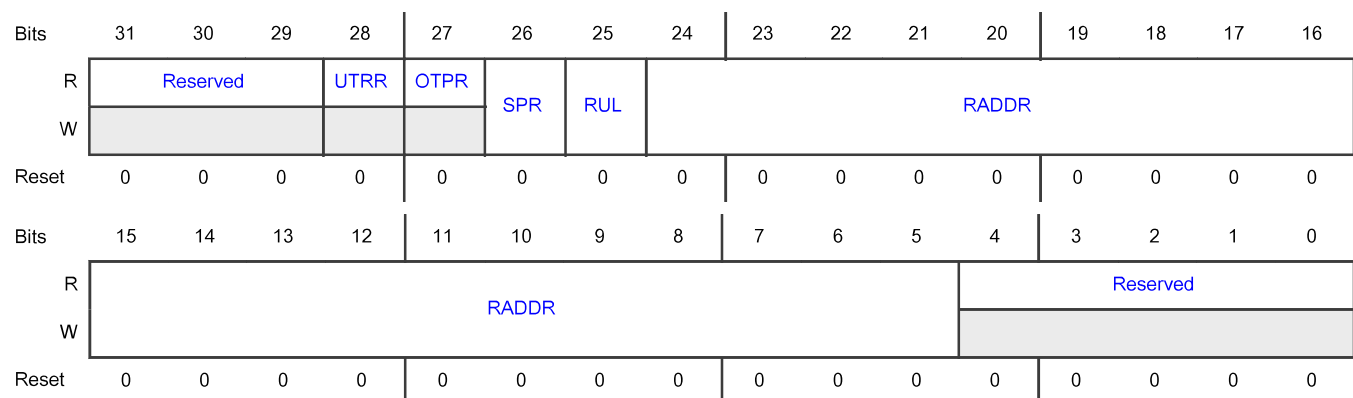


Table continued from the previous page...

Field	Function
4-0	Reserved
—	

40.6.5 Write Address Logging (WADR)

Offset

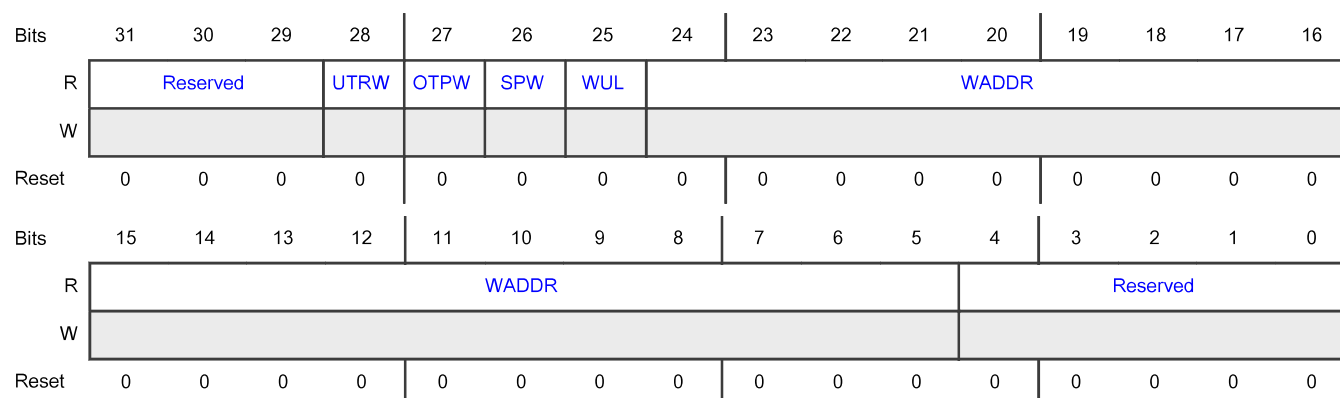
Register	Offset
WADR	10h

Function

Provides status for the following aspects of write operations:

- Error
- Correction

Diagram



Fields

Field	Function
31-29	Reserved
—	
28	UTest Row Write Select Qualifier
UTRW	Works with WADDR to indicate whether a UTest row region was selected for a write operation. F16ASM preserves the UTRW value until you clear the WRE and WCE flags in Module Configuration Status (MCRS) .

Table continues on the next page...

Table continued from the previous page...

Field	Function
	0b - Not selected 1b - Selected
27 OTPW	OTP Write Select Qualifier Works with WADDR to indicate whether an OTP region was selected for a write operation. F16ASM preserves the OTPW value until you clear the WRE and WCE flags in Module Configuration Status (MCRS) . 0b - Not selected 1b - Selected
26 SPW	Small Partition Write Qualifier Indicates which partition size to use with WADDR. F16ASM preserves the SPW value until you clear the WRE and WCE flags in Module Configuration Status (MCRS) . 0b - Large 1b - Small
25 WUL	Write Upper Lower Qualifier Indicates whether WADDR contains a lower or upper address. F16ASM preserves the WUL value until you clear the WRE and WCE flags in Module Configuration Status (MCRS) . 0b - Lower 1b - Upper
24-5 WADDR	Write Address Indicates the page address of the first written MRAM address that failed to match during verify read. For quad page write, the lowest failed address is reported. F16ASM preserves the captured write address until you clear the WRE and WCE flags in Module Configuration Status (MCRS) .
4-0 —	Reserved

40.6.6 Partition Lock 0 (LOCK0)

Offset

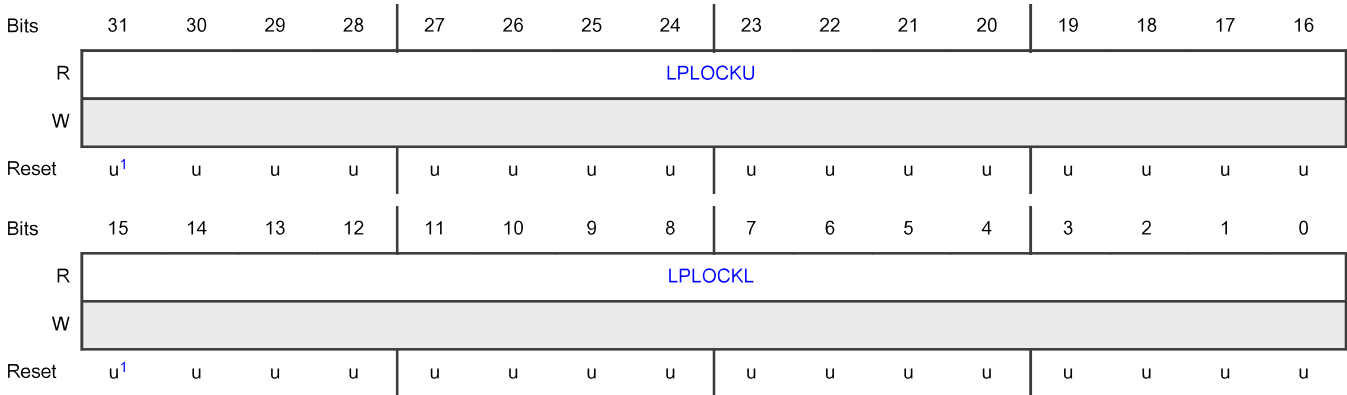
Register	Offset
LOCK0	14h

Function

Indicates the lock status of large partitions.

If you attempt to write to a locked partition, F16ASM sets the **MCRS[WRP]** flag.

Diagram



1. This reset value is chip-specific. See the chip-specific F16ASM information for the reset value on this chip. The MRAMC chapter has more information.

Fields

Field	Function
31-16 LPLOCKU	Large Partition Write Lock Upper Indicates the lock status of each upper partition. Each bit in this field corresponds to a partition. The value of each bit indicates whether the MRAMC inputs have locked that partition for writing: 0: Not locked 1: Locked
15-0 LPLOCKL	Large Partition Write Lock Lower Indicates the lock status of each lower partition. Each bit in this field corresponds to a partition. The value of each bit indicates whether the MRAMC inputs have locked that partition for writing: 0: Not locked 1: Locked

40.6.7 Partition Lock 1 (LOCK1)

Offset

Register	Offset
LOCK1	18h

Table continued from the previous page...

Field	Function
	1b - Locked
5 OLOCKU	Large Partition OTP Write Lock Upper Indicates whether the MRAMC inputs have locked the large upper OTP partition for writing. 0b - Not locked 1b - Locked
4 OLOCKL	Large Partition OTP Write Lock Lower Indicates whether the MRAMC inputs have locked the large lower OTP partition for writing. 0b - Not locked 1b - Locked
3-2 SPLOCKU	Small Partition Write Lock Upper Indicates the lock status of each upper small partition. Each bit in this field corresponds to a partition. The value of each bit indicates whether the MRAMC inputs have locked that partition for writing: 0: Not locked 1: Locked See chip-specific section to know the lock status behavior of the device.
1-0 SPLOCKL	Small Partition Write Lock Lower Indicates the lock status of each lower small partition. Each bit in this field corresponds to a partition. The value of each bit indicates whether the MRAMC inputs have locked that partition for writing: 0: Not locked 1: Locked See chip-specific section to know the lock status behavior of the device.

40.6.8 UTest 0 (UT0)

Offset

Register	Offset
UT0	1Ch

Function

Controls various aspects of UTest operation.

Table continued from the previous page...

Field	Function												
	• The array-integrity-check operation finishes. • You wrote 1 to AIBPE to enable breakpoints. • You requested an abort. • You requested a suspend. After AID becomes 1, you can examine the MISR (see UTest Multiple Input Signature n (UMISR0 - UMISR8) and UTest Multiple Input Signature 9 (UMISR9)). The following timing specifications govern how quickly F16ASM changes the value of AID. See the chip data sheet for the specification values. <table><tr><th>Event</th><th>AID change</th><th>How quickly</th></tr><tr><td>After you write 1 to AIE to enable an array-integrity check</td><td>1 → 0</td><td>T_{aistart}</td></tr><tr><td>After you write 0 to AIE to abort the operation</td><td>0 → 1</td><td>T_{aistop}</td></tr><tr><td>After you write 1 to AISUS to suspend the operation</td><td>0 → 1</td><td>T_{aistop}</td></tr></table> 0b - Ongoing 1b - Complete	Event	AID change	How quickly	After you write 1 to AIE to enable an array-integrity check	1 → 0	T _{aistart}	After you write 0 to AIE to abort the operation	0 → 1	T _{aistop}	After you write 1 to AISUS to suspend the operation	0 → 1	T _{aistop}
Event	AID change	How quickly											
After you write 1 to AIE to enable an array-integrity check	1 → 0	T _{aistart}											
After you write 0 to AIE to abort the operation	0 → 1	T _{aistop}											
After you write 1 to AISUS to suspend the operation	0 → 1	T _{aistop}											

40.6.9 UTest Multiple Input Signature n (UMISR0 - UMISR8)

Offset

For n = 0 to 8:

Register	Offset
UMISRn	20h + (n × 4h)

Function

Works with [UTest Multiple Input Signature 9 \(UMISR9\)](#) to specify a 289-bit signature ([MISR](#)) from an array-integrity event. See [Multiple input signature register \(MISR\)](#) for details of the MISR functionality.

Data and ECC captured in the MISR are after the ECC logic and represent corrected data (if required).

The entire MISR is distributed among the UMISR registers as follows:

Table 207. MISR distribution among UMISR registers

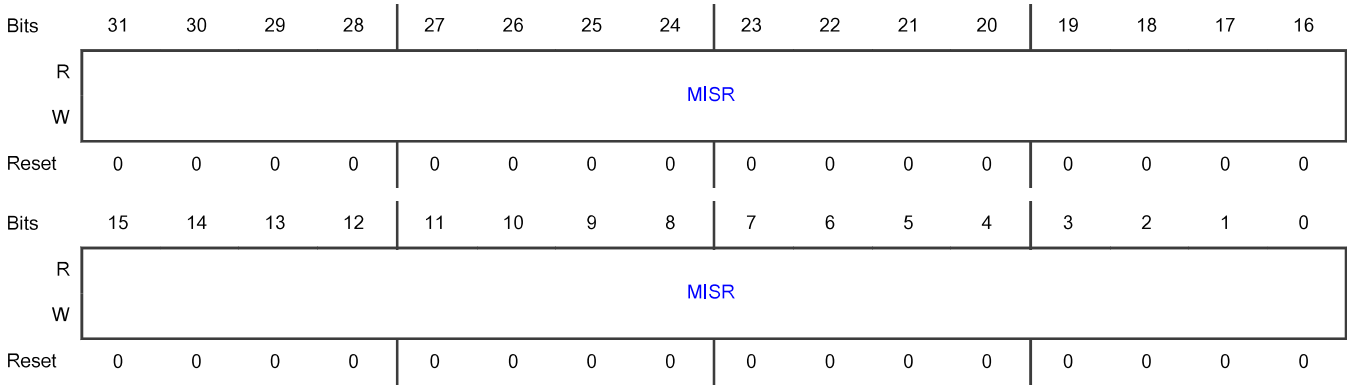
Register	MISR bits
UMISR0	31–0
UMISR1	63–32
UMISR2	95–64

Table continues on the next page...

Table 207. MISR distribution among UMISR registers (continued)

Register	MISR bits
UMISR3	127–96
UMISR4	159–128
UMISR5	191–160
UMISR6	223–192
UMISR7	255–224
UMISR8	287–256
UMISR9	288

Diagram



Fields

Field	Function
31-0	MISR Portion
MISR	Specifies part of the MISR as indicated in Table 207 .

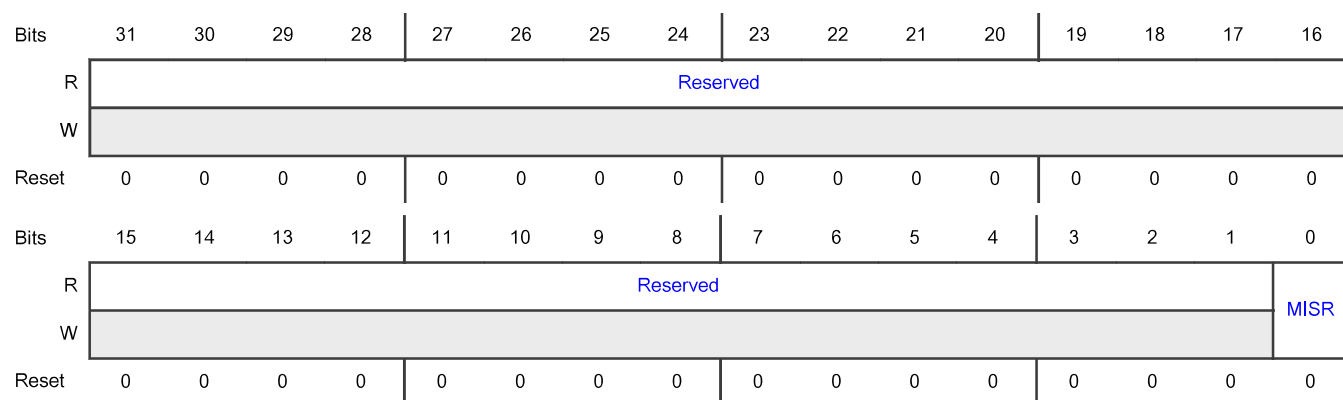
40.6.10 UTest Multiple Input Signature 9 (UMISR9)

Offset

Register	Offset
UMISR9	44h

Function

Works with [UTest Multiple Input Signature n \(UMISR0 - UMISR8\)](#) to specify a 289-bit signature (MISR) from an array-integrity event.

Diagram**Fields**

Field	Function
31-1 —	Reserved
0 MISR	MISR Portion Specifies part of the MISR as indicated in Table 207 .

40.6.11 UTest Data n (UD0 - UD7)**Offset**

For n = 0 to 7:

Register	Offset
UDn	48h + (n × 4h)

Function

Allows you to check the ECC logic.

With this register, you can input data bits into the ECC logic, then read them by performing a read operation or an array-integrity check.

The entire collection of data bits is distributed among the UD registers as follows:

Table 208. ECC data-bit distribution among UD registers

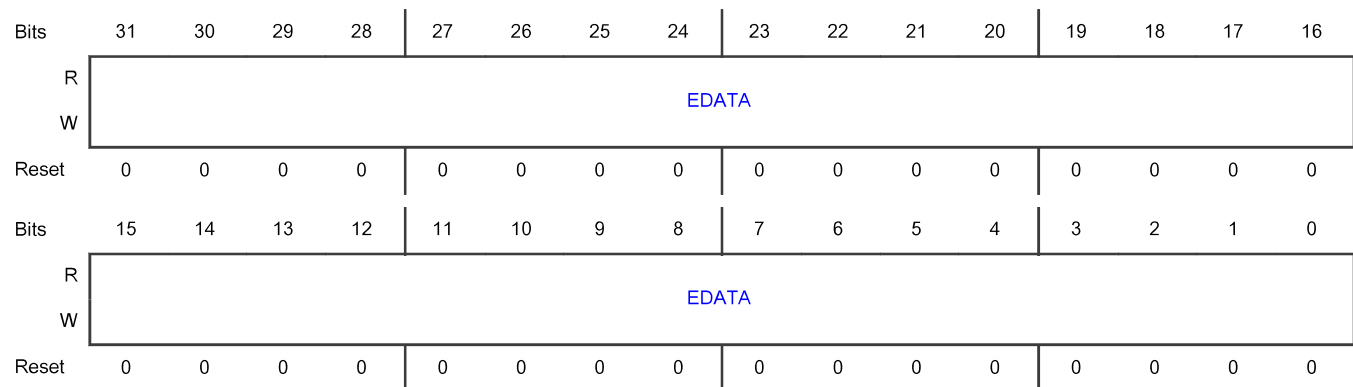
Register	ECC data bits
UD0	31–0
UD1	63–32
UD2	95–64
UD3	127–96

Table continues on the next page...

Table 208. ECC data-bit distribution among UD registers (continued)

Register	ECC data bits
UD4	159–128
UD5	191–160
UD6	223–192
UD7	255–224

Diagram



Fields

Field	Function
31-0 EDATA	ECC Data Portion Specifies part of the ECC data as indicated in Table 208 .

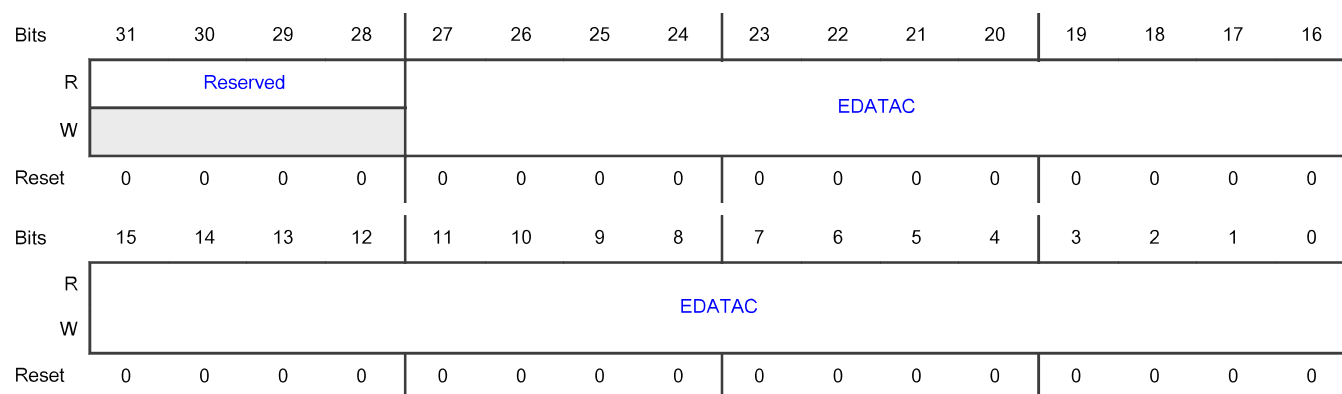
40.6.12 UTest Data 8 (UD8)

Offset

Register	Offset
UD8	68h

Function

Specifies the ECC data check bits of the page selected in [Read Address Logging \(RADR\)](#).

Diagram**Fields**

Field	Function
31-28 —	Reserved
27-0 EDATAC	ECC Data Check Bits

40.6.13 UTest Data n (UD9 - UD16)**Offset**

For n = 9 to 16:

Register	Offset
UDn	48h + (n × 4h)

Function

Allows you to check the EDC-after-ECC logic.

With this register, you can input data bits into the EDC-after-ECC logic, then read them by performing a read operation or an array-integrity check.

The entire collection of data bits is distributed among the UD registers as follows:

Table 209. EDC-after-ECC data-bit distribution among UD registers

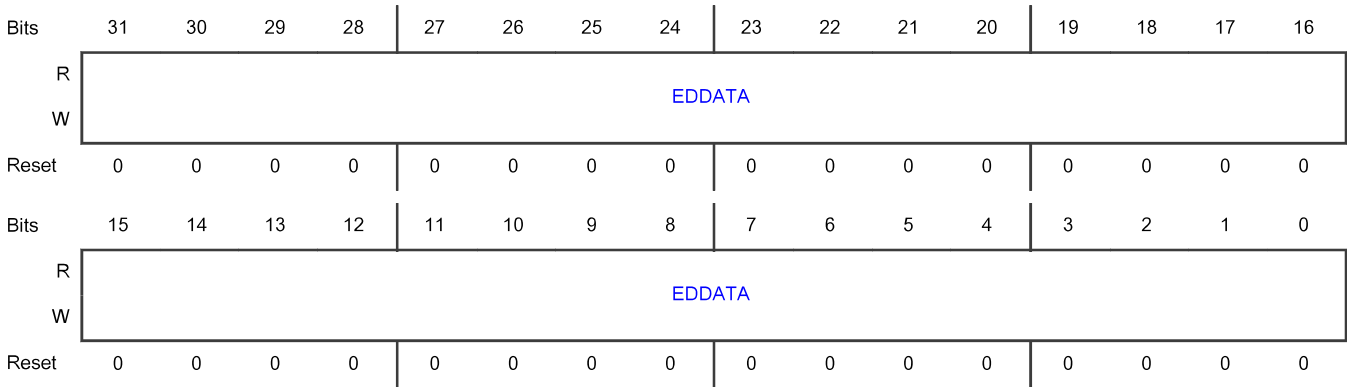
Register	EDC-after-ECC data bits
UD9	31–0
UD10	63–32
UD11	95–64
UD12	127–96

Table continues on the next page...

Table 209. EDC-after-ECC data-bit distribution among UD registers (continued)

Register	EDC-after-ECC data bits
UD13	159–128
UD14	191–160
UD15	223–192
UD16	255–224

Diagram



Fields

Field	Function
31-0 EDDATA	EDC-After-ECC Data Portion Specifies part of the ECC data as indicated in Table 209 .

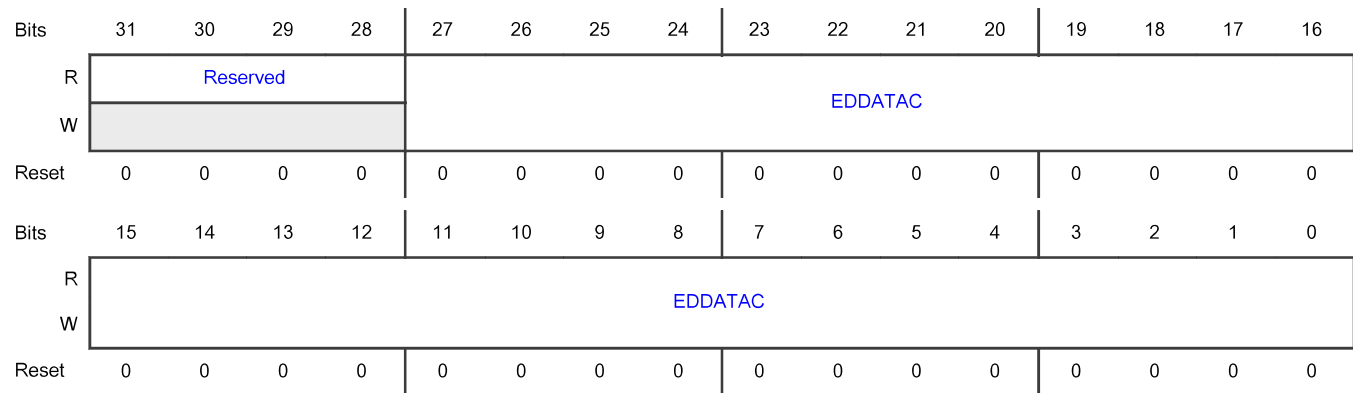
40.6.14 UTest Data 17 (UD17)

Offset

Register	Offset
UD17	8Ch

Function

Specifies the EDC-after-ECC data check bits of the page selected in [Read Address Logging \(RADR\)](#).

Diagram**Fields**

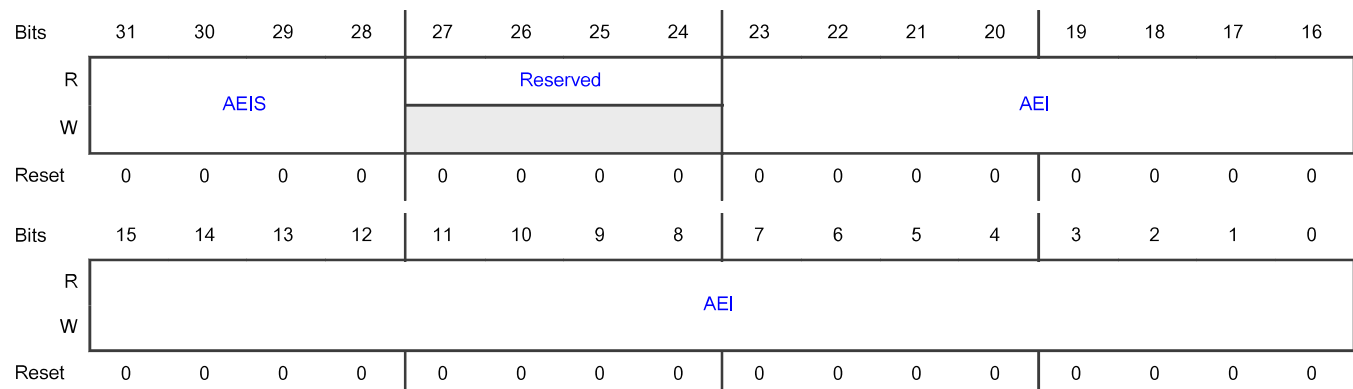
Field	Function
31-28 —	Reserved
27-0 EDDATAC	EDC-after-ECC Data Check Bits

40.6.15 UTest Address 0 (UA0)**Offset**

Register	Offset
UA0	94h

Function

Works with [UTest Address 1 \(UA1\)](#) to control address-encode inversion.

Diagram

Fields

Field	Function
31-28 AEIS	Address Encode Invert Select Selects which group of encodes (in AEI) to invert. Each bit within AEIS represents a group of encodes. You can select one to four groups. If you don't select any groups, F16ASM does not invert any encodes (AEI).
27-24 —	Reserved
23-0 AEI	Address Encode Invert Inverts bits that the address-encode compare logic uses, thus forcing a miscompare. You can use this behavior to check the address-encode logic. Performing array reads or array integrity check on the page selected in Read Address Logging (RADR) allows the address-encode invert to be active. AEI combines with UA1[AEIC] to represent the internal width of the address to the memory that is used for the address-encode comparison. When at least one bit in AEIS is 1 and you write 1 to any of the bits in the AEI fields, or you write a 1 to any of the bits in the UA1[AEIC] fields, F16ASM flags an AEE error when causing the error injection.

40.6.16 UTest Address 1 (UA1)

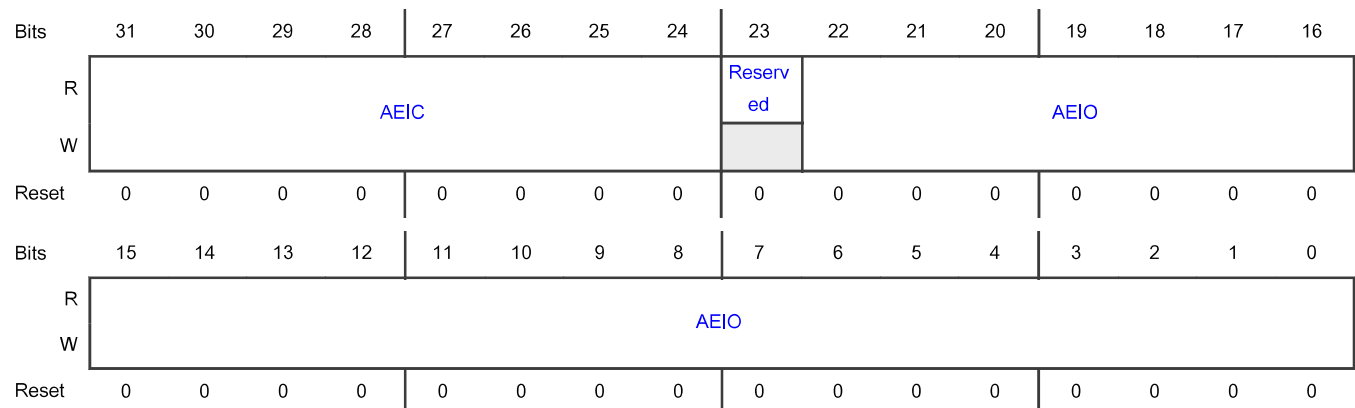
Offset

Register	Offset
UA1	98h

Function

Works with [UTest Address 0 \(UA0\)](#) to control address-encode inversion.

Diagram



Fields

Field	Function
31-24 AEIC	Address Encode Invert Common Inverts bits that the address-encode compare logic uses, thus forcing a miscompare. You can use this behavior to check the address-encode logic. Performing array reads or array integrity check on the page selected in allows the address-encode invert to be active. AEIC combines with UA0[AEI] to represent the internal width of the address to the memory that is used for the address-encode comparison. When at least one bit in AEIS is 1, and you write 1 to any of the bits in the AEI fields, or you write a 1 to any of the bits in the UA1[AEIC] fields, F16ASM flags an AEE error when causing the error injection.
23 —	Reserved
22-0 AEIO	Address Encode Invert Output Inverts address-output signals to the MRAMC. AEIO reflects: • The page address (MRAMC page address bits 24–5) • The small-partition select • The OTP select • The UTest row select When you write 1 to a bit in this field, F16ASM inverts the corresponding address-output signal to the MRAMC. You can use AEIO with the address-encode logic check to control this inversion feature (aligning with Read Address Logging (RADR)). You can write 1 to multiple bits; this causes multiple signals to invert. If an address-encode error occurs during normal user-mode operation, the page address provided to the MRAMC is inverted.

40.7 Glossary

CMMU	Common MRAM management unit, which provides read and write logic control for the MRAM
MISR	Multiple input signature register
MRAM	Magnetoresistive random-access memory
MRAMC	MRAM controller
OTP	One-time program
RMW	Read-modify-write operation
Tri-read	An F16ASM feature that allows you to read from up to three partitions in parallel